Fiche de préparation

Date de la séance : ……			Durée de la séance : **5 heures**					
Filière : **Génie électrique – TS**	1ère année :		2ème année		3ème année		Groupe : **GE 105**	
Module : **Logique combinatoire et séquentielle**								
Objectif de la séance : **À l'issue de la séance, le stagiaire sera capable de :** – expliquer la notion de base et de poids, et écrire un nombre en binaire, octal, décimal et hexadécimal ; – convertir un nombre d'une base à une autre (pondération, divisions successives, regroupements) ; – effectuer une addition binaire et une soustraction par complément à 2 ; – utiliser les codes BCD, Gray et ASCII, et le bit de parité.								

Introduction		**Durée**
Rappel	Première séance du module : rappel des prérequis, par questions-réponses : – puissances : $10^0 = 1$, $10^1 = 10$, $10^2 = 100$… ; $2^0 = 1$, $2^1 = 2$, $2^2 = 4$ … $2^{10} = 1024$; – division euclidienne (quotient et reste) ; – notion de signal logique (0 / 1 ; contact ouvert / fermé ; 0 V / 24 V) vue en automatisme.	10 min
Elément de motivation	Observation : un automate affiche l'adresse « %MW16#00FF », un codeur rotatif délivre « 0110 », un afficheur 7 segments montre « 7 », un variateur indique le code défaut « 3E ». Question : « Que signifient ces nombres ? Comment l'automate, qui ne connaît que 0 et 1, représente-t-il un nombre, une lettre, une position ? » Message : la numération et les codes sont la langue de tous les systèmes numériques (automates, variateurs, compteurs, capteurs) que le technicien installe et dépanne.	10 min
Plan de la Séance	**I. Les systèmes de numération** 1. Base, poids, bit et octet 2. Décimal, binaire, octal, hexadécimal 3. Conversion vers le décimal 4. Conversion du décimal vers une base 5. Conversions directes binaire ↔ octal / hexadécimal **II. Arithmétique binaire** 6. Addition binaire et complément à 2 **III. Les codes** 7. BCD, Gray, ASCII, bit de parité Cinq exercices d'application.	5 min

Stratégies pédagogiques	**Développement**	**Durée**
Exposé interactif Décomposition d'un nombre décimal au tableau	**I. LES SYSTÈMES DE NUMÉRATION** ***1. Base, poids, bit et octet*** Un système de numération de **base B** utilise **B symboles** (de 0 à B – 1). Chaque chiffre a un **poids** qui dépend de son **rang** : le rang 0 (à droite) vaut $B^0 = 1$, le rang 1 vaut B^1, le rang 2 vaut B^2… Exemple en base 10 : $3\,527 = 3 \times 10^3 + 5 \times 10^2 + 2 \times 10^1 + 7 \times 10^0$. – **bit** (binary digit) : chiffre binaire, 0 ou 1 ; **quartet** : 4 bits ; **octet** : 8 bits ; **mot** : 16 bits (selon le système) ; – **MSB** : bit de poids le plus fort (à gauche) ; **LSB** : bit de poids le plus faible (à droite) ; – avec **n bits**, on forme $\mathbf{2^n}$ combinaisons, de 0 à $\mathbf{2^n - 1}$ (8 bits : 256 valeurs, de 0 à 255). Notation de la base : $(1011)_2$ ou 1011b ; $(157)_8$; $(2AF)_{16}$, 2AFh ou 16#2AF (notation des automates).	20 min
Exposé + tableau de correspondance des 16 premiers nombres construit avec les stagiaires	***2. Décimal, binaire, octal, hexadécimal*** – **Décimal** (base 10) : chiffres 0 à 9 ; c'est notre système courant. – **Binaire** (base 2) : chiffres 0 et 1 ; c'est le langage des circuits numériques (2 états). – **Octal** (base 8) : chiffres 0 à 7 ; un chiffre octal = 3 bits. – **Hexadécimal** (base 16) : chiffres 0 à 9 puis **A = 10, B = 11, C = 12, D = 13, E = 14, F = 15** ; un chiffre hexadécimal = 4 bits. Il permet d'écrire les nombres binaires de façon compacte (1 octet = 2 chiffres hexadécimaux). Tableau à connaître (décimal / binaire 4 bits / hexa) : 0 = 0000 = 0 … 7 = 0111 = 7 ; 8 = 1000 = 8 ; 9 = 1001 = 9 ; 10 = 1010 = A ; 11 = 1011 = B ; 12 = 1100 = C ; 13 = 1101 = D ; 14 = 1110 = E ; 15 = 1111 = F.	20 min
Exposé + méthode écrite au tableau	***3. Conversion d'une base B vers le décimal*** On multiplie chaque chiffre par le **poids de son rang** et on additionne (**somme pondérée**) : $(N)_B = a_n \times B^n + \ldots + a_1 \times B^1 + a_0 \times B^0$ Exemple : $(1101)_2 = 1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 = \mathbf{13}$. Astuce pour le binaire : écrire au-dessus de chaque bit son poids (… 128, 64, 32, 16, 8, 4, 2, 1) et	15 min

	additionner les poids des bits à 1.	
Exercice résolu par le formateur pour a), puis travail individuel pour b) et c)	***Exercice 1 : conversion vers le décimal*** Énoncé : convertir en décimal : a) $(1011\ 0110)_2$; b) $(157)_8$; c) $(2AF)_{16}$. **Solution :** a) Poids : 128, 64, 32, 16, 8, 4, 2, 1 ; bits à 1 : 128 + 32 + 16 + 4 + 2 = **182**. b) $1 \times 8^2 + 5 \times 8^1 + 7 \times 8^0 = 64 + 40 + 7 =$ **111**. c) $2 \times 16^2 + A \times 16^1 + F \times 16^0 = 2 \times 256 + 10 \times 16 + 15 \times 1 = 512 + 160 + 15 =$ **687**.	15 min
Exposé + démonstration de la méthode des divisions successives au tableau	***4. Conversion du décimal vers une base B*** **Méthode des divisions successives** : on divise le nombre par B, puis le quotient par B, et ainsi de suite jusqu'à obtenir un quotient nul. Le résultat s'écrit avec les **restes lus de bas en haut** (le dernier reste est le MSB). Exemple : 13 en binaire : 13 / 2 = 6 reste 1 ; 6 / 2 = 3 reste 0 ; 3 / 2 = 1 reste 1 ; 1 / 2 = 0 reste 1 ⇒ $(13)_{10} =$ **$(1101)_2$**. Autre méthode pour le binaire (**soustraction des puissances de 2**) : on retire la plus grande puissance de 2 possible et on met un 1 à son rang, puis on recommence avec le reste.	20 min
Travail individuel, puis correction au tableau	***Exercice 2 : conversion du décimal*** Énoncé : a) convertir 200 en binaire et en hexadécimal ; b) convertir 1000 en hexadécimal puis en binaire ; c) combien de bits faut-il au minimum pour coder 1000 ? **Solution :** a) 200 / 2 : restes successifs 0, 0, 0, 1, 0, 0, 1, 1 (lus de bas en haut) ⇒ **$(1100\ 1000)_2$**. Vérification : 128 + 64 + 8 = 200 ✓. En hexa : 200 / 16 = 12 reste 8 ; 12 / 16 = 0 reste 12 (C) ⇒ **$(C8)_{16}$**. b) 1000 / 16 = 62 reste 8 ; 62 / 16 = 3 reste 14 (E) ; 3 / 16 = 0 reste 3 ⇒ **$(3E8)_{16}$** ⇒ 3 = 0011, E = 1110, 8 = 1000 ⇒ **$(11\ 1110\ 1000)_2$**. Vérification : $3 \times 256 + 14 \times 16 + 8 = 768 + 224 + 8 = 1000$ ✓. c) $2^9 = 512 < 1000 < 2^{10} = 1024$ ⇒ **10 bits**.	15 min
—	**PAUSE**	15 min
Exposé + exemple au tableau	***5. Conversions directes binaire ↔ octal / hexadécimal*** – **Binaire → octal** : regrouper les bits **par 3** en partant de la droite, puis remplacer chaque groupe par son chiffre octal. – **Binaire → hexadécimal** : regrouper les bits **par 4** en partant de la droite (compléter à gauche par des 0), puis remplacer chaque groupe par son chiffre hexa. – **Octal ou hexa → binaire** : remplacer chaque chiffre par ses 3 (ou 4) bits. Exemple : $(1111\ 0010)_2$ = **$(F2)_{16}$** ; $(F2)_{16} = 15 \times 16 + 2 = 242$.	15 min
Travail individuel, puis correction orale	***Exercice 3 : conversions directes*** Énoncé : convertir $(1101011101)_2$ en octal et en hexadécimal, puis vérifier en passant par le décimal. Convertir $(7F)_{16}$ en binaire. **Solution :** Octal : 1 | 101 | 011 | 101 ⇒ **$(1535)_8$**. Hexa : 11 | 0101 | 1101 ⇒ 3 | 5 | D ⇒ **$(35D)_{16}$**. Vérification : $(1535)_8 = 512 + 5 \times 64 + 3 \times 8 + 5 = 512 + 320 + 24 + 5 =$ **861** ; $(35D)_{16} = 3 \times 256 + 5 \times 16 + 13 = 768 + 80 + 13 =$ **861** ✓. $(7F)_{16} = 0111\ 1111 =$ **$(0111\ 1111)_2$** = 127.	15 min
Exposé + démonstration pas à pas au tableau	**II. ARITHMÉTIQUE BINAIRE** ***6. Addition binaire et complément à 2*** **Addition** : 0 + 0 = 0 ; 0 + 1 = 1 ; 1 + 1 = 0 avec une **retenue** de 1 ; 1 + 1 + 1 = 1 avec retenue 1. **Nombres négatifs : complément à 2** (sur un nombre de bits fixé, par ex. 8 bits) : 1) écrire le nombre positif en binaire ; 2) **inverser tous les bits** (complément à 1) ; 3) **ajouter 1**. Exemple : –5 sur 8 bits : 5 = 0000 0101 → inversion 1111 1010 → + 1 = **1111 1011**. Le MSB indique le **signe** (0 = positif, 1 = négatif). Sur 8 bits, on code de –128 à +127. **Soustraction** : A – B = A + (complément à 2 de B) ; on ignore la retenue qui sort du nombre de bits.	25 min
Travail en binôme, puis correction au tableau	***Exercice 4 : addition et soustraction binaires*** Énoncé : a) effectuer $(1011)_2 + (0110)_2$ et vérifier en décimal ; b) calculer 12 – 5 en binaire sur 8 bits par le complément à 2 ; c) écrire –20 en complément à 2 sur 8 bits. **Solution :** a) 1011 + 0110 = **10001** ; vérification : 11 + 6 = 17 = 16 + 1 ✓. b) 12 = 0000 1100 ; –5 = 1111 1011 ; somme = **1** 0000 0111 ; on ignore la retenue sortante ⇒ 0000 0111 = **7** ✓. c) 20 = 0001 0100 → inversion 1110 1011 → + 1 = **1110 1100**.	15 min
Exposé +	**III. LES CODES**	25 min

démonstration : codeur absolu (Gray), afficheur 7 segments (BCD), table ASCII	***7. BCD, Gray, ASCII et bit de parité*** – **BCD (DCB, décimal codé binaire)** : chaque **chiffre décimal** est codé séparément sur **4 bits** (0000 à 1001). Les combinaisons 1010 à 1111 sont **interdites**. Utilisé pour les afficheurs et les roues codeuses. Ex. : 59 = 0101 1001 (≠ binaire naturel 111011). – **Code Gray (binaire réfléchi)** : **un seul bit change** entre deux valeurs successives ; il évite les erreurs de lecture des **codeurs absolus** de position. Binaire → Gray : le MSB est recopié, puis chaque bit de Gray = OU exclusif de deux bits binaires voisins. Gray → binaire : le MSB est recopié, puis chaque bit binaire = OU exclusif du bit binaire précédent et du bit Gray suivant. – **ASCII** : code de 7 bits qui représente les **caractères** (lettres, chiffres, ponctuation, commandes) : « A » = 65 = 41h ; « a » = 97 = 61h ; « 0 » = 48 = 30h. – **Bit de parité** : bit ajouté pour détecter une erreur de transmission ; en **parité paire**, le nombre total de 1 (bit de parité compris) doit être pair.	
Travail individuel, puis correction collective	***Exercice 5 : utiliser les codes*** Énoncé : a) coder 395 en BCD et en binaire naturel ; b) décoder le BCD 0110 1000 0111 ; c) 1010 est-il un code BCD valide ? d) convertir $(1011)_2$ en Gray et le Gray 1101 en binaire ; e) donner le code ASCII de « A » en binaire et le bit de parité paire ; f) un codeur absolu doit distinguer 360 positions : combien de bits ? **Solution :** a) BCD : 3 → 0011, 9 → 1001, 5 → 0101 ⇒ **0011 1001 0101** ; binaire : **1 1000 1011** (256 + 128 + 8 + 2 + 1). b) 0110 = 6, 1000 = 8, 0111 = 7 ⇒ **687**. c) 1010 = 10 > 9 ⇒ **non valide** en BCD. d) Binaire 1011 → Gray : 1, 1⊕0 = 1, 0⊕1 = 1, 1⊕1 = 0 ⇒ **1110**. Gray 1101 → binaire : 1, 1⊕1 = 0, 0⊕0 = 0, 0⊕1 = 1 ⇒ **1001**. e) « A » = 65 = **100 0001** ; deux bits à 1 (nombre pair) ⇒ bit de parité paire = **0**. f) $2^8 = 256 < 360 \leq 2^9 = 512$ ⇒ **9 bits** (codés en Gray).	15 min

Conclusion		**Durée**
Synthèse	Synthèse construite avec les stagiaires (questions orales) : – Base B, poids B^{rang} ; n bits → 2^n valeurs (0 à $2^n - 1$) ; MSB / LSB. – Vers le décimal : somme pondérée ; depuis le décimal : divisions successives (restes de bas en haut). – Binaire ↔ octal : groupes de 3 bits ; binaire ↔ hexa : groupes de 4 bits. – Complément à 2 : inverser les bits puis ajouter 1 ; A – B = A + C2(B). – BCD : 4 bits par chiffre ; Gray : un seul bit change ; ASCII : caractères sur 7 bits ; parité : détection d'erreur.	15 min
Evaluation	Évaluation écrite individuelle (réponses attendues entre crochets) : ***Exercice A*** Convertir $(11010)_2$ et $(3C)_{16}$ en décimal, puis 45 en binaire. [$(11010)_2 = 16 + 8 + 2 = 26$; $(3C)_{16} = 3 \times 16 + 12 = 60$; $45 = 32 + 8 + 4 + 1 = (101101)_2$] ***Exercice B*** Coder 45 en BCD. Convertir le binaire 0110 en code Gray. Pourquoi utilise-t-on le Gray dans les codeurs ? [0100 0101 ; Gray 0101 ; un seul bit change entre deux positions voisines, pas d'erreur de lecture] ***Exercice C*** Écrire –7 en complément à 2 sur 8 bits et calculer 9 – 7 en binaire. [7 = 0000 0111 → 1111 1000 → + 1 = 1111 1001 ; 0000 1001 + 1111 1001 = 1 0000 0010 → 0000 0010 = 2 ✓]	25 min
Prochaine séance	Suite du module : variables et fonctions logiques, portes logiques (ET, OU, NON, NAND, NOR, OU exclusif) et tables de vérité. Travail demandé : construire le tableau des nombres de 0 à 31 en décimal, binaire, hexadécimal, BCD et Gray.	5 min